

Makara Ramoabi

20240045

C++

1. C++ OOP Scenario

In a C++ Object-Oriented Programming (OOP) scenario, a bank account system can be modeled using classes and objects. Each bank account is represented as an object created from a BankAccount class. The class contains data members such as account number, account holder name, and balance. These data members store the state of the account. The class also includes member functions like deposit(), withdraw(), and checkBalance() that define the behavior of the account. When a customer deposits money, the deposit() function updates the balance securely. The withdraw() function ensures that the user cannot withdraw more money than available. By using OOP, multiple bank accounts can be created from the same class while keeping their data separate. This approach makes the program organized, reusable, and easier to maintain. It also reflects real-world banking operations accurately.

2. OOP Principle Research

Encapsulation is an important principle of Object-Oriented Programming that involves bundling data and methods into a single unit called a class. In encapsulation, data members are usually declared as private, meaning they cannot be accessed directly from outside the class. Instead, public member functions are used to access or modify the data safely. This helps protect sensitive data from accidental changes and misuse. Encapsulation improves program security, maintainability, and flexibility. If the internal implementation changes, the external code does not need to be modified. Overall, encapsulation helps create well-structured and reliable C++ programs.

3. C++ Class

C++ Code

Copy code

Cpp

```
#include <iostream>
```

```
using namespace std;
```

```
class Rectangle {
```

```
private:
```

```
    int length;
```

```
    int width;
```

```
public:
```

```
    void setValues(int l, int w) {
```

```
        length = l;
```

```
        width = w;
```

```
    }
```

```
    int area() {
```

```
        return length * width;
```

```
    }
```

```
};
```

```
int main() {
```

```
    Rectangle rect;
```

```
    rect.setValues(5, 4);
```

```
    cout << "Area of rectangle: " << rect.area() << endl;
```

```
    return 0;
```

```
}
```

Sample Output:

Copy code

Area of rectangle: 20

Reflection

Creating and running the Rectangle class helped me understand how classes and objects work in C++. The class defines properties and behavior, while the object uses them. By keeping the data members private, the program follows encapsulation, which improves safety. The use of member functions makes the code more organized and readable. Running the program showed how objects interact with functions to produce results. This exercise improved my confidence in writing basic C++ OOP programs and understanding how real-world objects can be represented using classes.

4. C++ OOP Diagram

```
+-----+
|  Rectangle  |
+-----+
| - length : int |
| - width  : int |
+-----+
| + setValues() |
| + area ()     |
+-----+
```

Explanation

This diagram represents a simple C++ class structure. The Rectangle class contains private data members length and width, which store the dimensions of the rectangle. These variables are hidden from direct access to ensure data protection. The public functions set-values () and area ()

allow controlled interaction with the data. This structure demonstrates encapsulation, where data and methods are combined into one unit. Class diagrams help visualize program structure and make it easier to understand relationships between data and functions.